Qwertie's SNES Documentation Plus DMA Revision 6 (2.1)

This document is **more** than just a regurgitation (how is that spelled?) of the worthless SNES manual and Yoshi Doc. I wrote it all by myself from scratch based on experiences writing my emulator.

This repeats most everything contained in Yoshi's doc, but expands on it in much greater detail, in better and less ambiguous English. Since it is in HTML, it's got plenty of cross-references. I, myself, read this document when I can't remember something about the SNES. It is designed as a sort of compromise, intended to be read by both SNES developers and emulator authors.

It used to cover only graphics, but now also covers DMA, HDMA, Indirect HDMA, joypad registers (including the old-style reading method, which is not covered by Yoshi's doc), and Windowing.

Note: Unless otherwise stated, all 16-bit number values in this document should be assumed to be in big-endian (is that what it's called?) format, where the least significant byte occupies the lower memory location and the high byte occupies the following memory location.

Table of Contents

SNES Graphics Terminology

- PPU: Picture processing unit. It is the thing that takes your SNES graphics data and turns it into an image on the TV screen.
- VRAM: Video RAM. 64K of memory where tiles and tile maps are stored
- Registers: Memory-mapped I/O ports used for sending data and commands to the PPU.
- OAM (also known as OBJ, object, or sprite info): Information about the sprites on the screen, or the memory that contains this information. OAM stands for "Object Attribute Memory".
- CG: Color palette data, or the memory that stores it. There are 256 palette entries which contain the 15-bit absolute color levels.
- SC: Screen. Usually refers to the tile map for a BG, which is stored in VRAM.
- BG: Background. There are up to 4 BGs, or "layers", of tiles that form the basis of SNES graphics. These BGs can be scrolled, and a special type of BG, in "Mode 7", can be scaled and rotated and stuff to look 3-D. Sometimes the term "plane" (scrolling plane, as opposed to color plane) is used (BG1=Plane 0, BG2=Plane 1, etc.)

- Sprites: Everybody knows what these are, come on. They are the little (well, they can be up to 64x64 pixels on the SNES) graphics usually used for individual moving objects in a game.
- Tile map (a.k.a. BG Matrix): A BG is made up of hundreds of tiles, which are simply little graphics of a certain size (8x8, and 16x16 which is made of 4 8x8 tiles) placed end-to-end. A tile map is a two-dimensional array which contains a tile number and other properties for each square on the BG. Note that, depending on the context, the word "tile" may refer to the two-byte entry in the tile map, or else data for an 8x8 graphic character.
- Character: the graphic data (pixels) for a tile.
- Priority: Attribute that determines, or helps to determine, the order in which BGs and sprites are drawn on the screen.

Types of PPU Memory

There are three memory areas where different kinds of graphics data are stored, as well as the registers (located in the reserved 65816 address space between $2000 and $5FFF) which set certain screen parameters.

The main data area is the VRAM, a 64 KB memory space that can be accessed with registers $2115, $2116, $2118, and $2139. This area is used for storing all the tiles used in your game, as well as the tile maps.

The second data area, the OAM, is used to store properties of the sprites. It includes information about position, size, priority, etc. There can be 128 objects maximum, and the memory is 544 bytes: the first 512 bytes have four bytes of information per sprite, and the last 32 bytes have two more bits of information. Two or more sprites can share the same set of tiles.

The third area is the CGRAM, where the palette data is stored. It is 512 bytes: two bytes, and fifteen bits, for each of the 256 on-screen colors. There are five bits for each primary color (red, green, and blue).

The SNES PPU Graphics Organization

Basics of BGs

In VRAM there is a two-dimensional array that is a map of the tiles on the screen. Depending on the setting of registers $2107 to $210A, this map may be 32x32, 32x64, 64x32 or 64x64 tiles in size (see a note in the register description about the format of maps larger than 32 tiles wide or high). Each of the entries in this map contain the following data:

```
High     Low          Legend->  c: Starting character (tile) number
vhopppcc cccccccc               h: horizontal flip  v: vertical flip
                                p: palette number   o: priority bit
```

The character number indexes into an "array of tiles" starting at a base VRAM location selected by registers $210B or $210C. The byte address in VRAM where the character data starts can be found using the following calculation:

address_of_character = (base_location_bits << 13) + (8 * color_depth * character_number);

This formula works in both 8x8 and 16x16 tile mode. For example, if base location 1 is selected, the color depth for the tile map is 4 bits (16 colors), and character number 1 is selected, the address will be (1<<13)+8*4*1 = 8224.

Note that the word address entered into the VRAM address register will be half of that, or 4112.

Except when using 256-color BGs, the palette bits determine what colors in the CGRAM will be mapped to the colors in the character. The number formed by the palette bits is multiplied by the number of colors in the BG to get the starting index in the color palette. For example, if palette 3 is selected in 16 color mode, the tile's colors will range from CG entries 48 to 63. However, entry #48 will be unused since this is mapped from color 0, which is always the 'transparent color'. Notice that, except in a 256-color mode, it is not possible to select colors

above 127. That's okay; the sprites use these colors. Also, except in Mode 0 (see register $2105), four color BGs may only use colors 1 to 31.

The horizontal and vertical flip bits, if set to 1, will cause the characters to be mirrored when shown on the screen, so that they are facing the opposite direction. When in 16x16 tile mode, the entire tile is flipped (pixel 0 is swapped with pixel 15) rather than the individual 8x8 sub tiles being flipped.

The priority bit has the effect of deciding whether a given tile is 'on top of' or behind other BGs and sprites. For more information on the drawing order, see bit 3 of register $2105.

Basics of sprites

All SNES sprites are 16 colors. The SNES can have two sizes of sprites on the screen at once; the two sizes are selected with bits 5-7 of register $2101. The character data for sprites is stored in VRAM, in the same format as BG tiles. Two or more sprites can share the same set of tiles. When the sprites are larger than 8x8, they are arranged in columns, followed by rows of 8x8 tiles. For example, a 32x32 sprite is stored like this:

```
Byte Offset  0     32    64     96     128   160   192    224    256    ... et cetera.
Tile Coord   (0,0) (8,0) (16,0) (24,0) (0,8) (8,8) (16,8) (24,8) (0,16) ... and so on.
```

Now that you've consumed the above information, realize that it is not completely correct. The SNES has one more display quirk that is thrown into the mix: all of the rows must be stored 16 tiles apart (i.e. 512 bytes, or 256 words.) This means that, if there was one 32x32 sprite in VRAM, it would have to be stored like this:

```
Offset  Y-coord. Tile Coord
0       0-7      (0, 0) (8, 0) (16, 0) (24, 0) <Unused--room for 12 more tiles>
512     8-15     (0, 8) (8, 8) (16, 8) (24, 8) <Unused--room for 12 more tiles>
1024    16-23    (0,16) (8,16) (16,16) (24,16) <Unused--room for 12 more tiles>
1536    24-31    (0,24) (8,24) (16,24) (24,24) <Unused>
```

In practice, the sprites are interleaved. In other words, when using 32x32 tiles, there would be 3 more sprites stored in that "unused" space. If the first sprite, shown in the above table, started at offset 0, the next sprite would start at offset 128 (the Character Number, in OAM, would be 4); the third sprite would start at offset 256 (Character #8), and the fourth sprite would start at offset 384 (Character #12). If a fifth sprite was desired, the pattern would repeat and it would be located at offset 2048 (Character #64).

Similarly, when using 16x16 sprites, there will be 8 sprites interleaved, and the ninth sprite will have to start after them at offset 1024. Finally, 64x64 sprites have only two sprites interleaved. It is also possible to format it such that larger sprites can be interleaved with smaller sprites, but that is somewhat confusing to try to explain.

In OAM there are two tables which control the position, size, mirroring, palette, and priority of sprites. This table has room for 128 entries; thus, the SNES can display up to 128 sprites on the screen at once (although I think a real SNES will overload and screw up its display when there are too many large sprites.) The first table has four bytes per sprite, and is formatted like this:

```
Byte 1    xxxxxxxx    x: X coordinate
Byte 2    yyyyyyyy    y: Y coordinate
Byte 3    cccccccc    c: starting character (tile) number    p: palette number
Byte 4    vhoopppc    v: vertical flip   h: horizontal flip  o: priority bits
          Note: the 'c' in byte 4 is the MOST significant bit in the 9-bit char #.
```

The second table is 32 bytes and has 2 bits for each sprite (each byte contains information for 4 sprites.) The lowest significant bits hold the information for the lower object numbers (for example, the least significant two bits of the first byte are for object #0.) Bit 0 (and 2, 4, 6) is the size toggle bit (see bits 5-7 of register $2101) and bit 1 (3, 5, 7) is the most ignificant bit of the X coordinate.

The vertical and horizontal flips work similarly to flips in the BGs; the entire sprite is flipped so that the leftmost pixel is swapped with the rightmost pixel, etc. To see the effect of the priority bits, see the description of register $2105. The palettes start at CG entry 128, so that palette 0 is colors 128 to 143, and palette 1 consists of colors 144 to 159.

The character number indexes into an array of 8x8 tiles starting at a base VRAM location selected by bits 0-2 of register $2101. The byte address in VRAM where the character data starts can be found using the following calculation:

```
address_of_character = (base_location_bits << 14) + (32 * character_number);
```

For example, if base location 1 is selected, and character number 1 is selected, the address will be 16384+32*1 = 16416. Note that the word address entered into the VRAM address register will be half of that, or 8208.

Notice that two or more sprites in the OAM table may have the same character number; if this is the case, they will look the same except that they can be mirrored independently and have different palettes.

Color palettes

Once the final color value is derived from the character data and 'palette number' for the sprite or BG, it is indexed into the CGRAM color palette array. There are 512 bytes of CGRAM, with each of the 256 colors using two bytes. Each of the palette entries is formatted like this:

```
?bbbbbgg gggrrrrr    ?: Unused and ignored  b: Blue intensity
                     g: Green intensity     r: Red intensity
```

(Notice this is backwards to the conventional RGB format.) To upload palette entries to CGRAM, select the color with register $2121, and then begin sending the bytes to register $2122.

The SNES Graphics Format

All SNES graphics (except in BG1 of Mode 7) are made up of Tiles, also called character data. The basic tile is 8x8 in size, and larger bitmaps are stored in the form of two-dimensional arrays of these small tiles. On most computers, bitmaps are stored in a packed format: the color bits are grouped together in the same byte (or word, or dword, for high-color displays.) However, SNES graphics are planar; that is, the each of the color bits are stored separately. Each color plane of the tile is stored like this:

```
byte 0:   top scanline   01234567
byte 2:                  01234567     right
byte 4:        left      01234567     side
byte 6:        side      01234567      of
byte 8:         of       01234567     tile
byte 10:       tile      01234567
byte 12:                 01234567  Note that left/right and top/bottom can
byte 14: bottom scanline 01234567  be reversed using the BG/sprite flip bits.
```

Planes 0 and 1 are stored first, followed by planes 2 and 3, etc. As you can see, the bytes in each plane are two bytes apart; the byte "in between" is the following plane. In other words, byte 0 stores the first eight pixels of plane 0 and byte 1 stores the first eight pixels of plane 1. If using more than 4 colors, the pattern repeats; byte 16 stores the first eight pixels of plane 2 and byte 17 stores the first eight pixels of plane 3.

When the tile is to be displayed on the screen, a bit is taken from each of the planes to form a 2-, 4-, or 8-bit value to index into the color palette. The first tile (plane 0) contains the least significant bit of the color, and the last tile contains the most significant bit.

Mode 7 Graphics Representation

In Mode 7, both the tile map and the tiles themselves are stored differently. In fact, the tile data and the graphic data are interleaved. The first byte (low byte) is an element of the tile map, and the second byte (high byte) is the color value of a pixel.

```
          First Byte              Second Byte
Bits      7  6  5  4  3  2  1  0  15 14 13 12 11 10 9  8
Contains  Tile Number             Graphics data
```

The Mode 7 screen data occupies all of the first 32 KB of VRAM: 16K is for graphics data, and 16K is for the tile numbers. The tile map format is simply the tile number with no other information, and takes one byte. The dimensions of the tile map are 128x128 (=16384 bytes). Unlike all other screen modes, the graphics data is a packed, linear format and each color value directly indexes into CGRAM. The tiles are still 8x8 pixels, and thus take 64 bytes per tile. There are 256 characters in total. The calculation to find which character to display is simply 64 * Tile_Number.

A variation of Mode 7, EXTBG (see bit 6 of register $2133), cuts the colors down to 128 and uses the most significant color bit as a priority bit.

Register Reference

This section may not be quite accurate. Please tell me if you find an error. W means writeable; R means readable. 2b means the register is one word in size; Db means that it is a one-byte register that must be written or read twice. In contrast to the Y0shi Doc, auto-incrementing registers such as $2122 are not considered Db registers, because they can be written any number of times (writing one byte will work just as well as writing two, or writing a hundred.)

Can anyone tell me what is supposed to happen when you read from a write register?

OAM Registers

Register $2101: OAM Size (1b/W)

```
sssnnbbb   s: Object size  n: name selection  b: base selection
           Size bit in OAM table:           0     1
           Bits of object size:    000      8x8   16x16
                                   001      8x8   32x32
                                   010      8x8   64x64
                                   011      16x16 32x32
                                   100      16x16 64x64
                                   101      32x32 64x64
                                   110,111  Unknown behavior
```

This register selects the location in VRAM where the character data is stored, and the size of sprites on the screen. The byte location of the character data can be found by shifting the b (base selection) bits left by 14. Note that this allows only four different locations in VRAM to put the sprite data; the high bit of the base selection should always be zero since only 64K of VRAM can be addressed.

I have no information on the name selection bits.

Register $2102/$2103: Address for accessing OAM (2b/W)

```
aaaaaaaa r??????m  a: low byte of OAM address
                   r: OAM priority rotation  m: OAM address MSB
```

This register selects the byte location to begin uploading (or downloading) data to OAM.

I'm sorta guessing, but I think that the priority rotation thing is applied by SNES games to keep sprites on the screen. That is, when there are too many sprites on the screen at once I believe the SNES will turn certain sprites off in order to reduce the load on the PPU. So I think the SNES takes the "a" bits, shifts them right one and the result is which sprite to re-activate. In the process another sprite gets turned off. Another possibility is that it doesn't matter what address you select, it simply picks any sprite that is off and turns it on, perhaps simultaneously turning off a sprite that has been on for a while.

Register $2104: Data write to OAM (1b/W)

```
dddddddd  d: byte to write to VRAM
```

This register writes a byte to OAM. After the byte is stored, the OAM address is incremented so that the next write or read will be to the following address.

Register $2138: Data read from OAM (1b/R)

```
dddddddd  d: byte that was read from OAM
```

After the byte is read, the OAM address is incremented so that the next write or read will be to the following byte.

Color Registers

Register $2121: Address for accessing CGRAM (1b/W)

```
aaaaaaaa a: CGRAM word address
```

This register selects the word location (byte address * 2) to begin uploading (or downloading) data to CGRAM. Click HERE for more information on color palettes.

Register $2122: Data write to CGRAM (1b/W)

```
dddddddd  d: byte to write to CGRAM
```

This register writes a byte to CGRAM. After the byte is stored, the CGRAM address is incremented so that the next write or read will be to the following byte.

Register $213B: Data read from CGRAM (1b/R)

```
dddddddd  d: byte that was read from CGRAM
```

After the byte is read, the CGRAM address is incremented so that the next write or read will be to the following byte.

VRAM Transfer/Address Registers

Register $2107-$210A: Tile map location (4*1B/W)

```
Bits in each of the registers:
aaaaaass  a: Tile map address  s: SC size: 00=32x32 01=64x32
                                           10=32x64 11=64x64
```

To calculate the byte location where the tile map starts, shift the a (address) bits left by 11 (multiply by 2048.) The SC size is the dimensions of the tile map; if using 8x8 tile mode, this allows BG dimensions of 256 or 512 pixels; if in 16x16 mode, the dimensions can be 512 or 1024 pixels. Note that, since there is only 64K of VRAM, the most significant bit must be zero.

When using a screen size wider than 32 tiles, the format is a little different than you might expect. When the width is 64 tiles, then rather than each line in the tile map extending to 128 bytes (instead of 64), there will actually be *two* tile maps, stored one right after the other in memory. The first tile map will contain the left 32 tiles (x coordinates 0 to 255, when using 8x8 tiles), and the next tile map will contain the right 32 tiles (x coordinates 256 to 511, when using 8x8 tiles. Setting the scroll register to 512, then, will be the same at setting it to zero.)

A note about using 16x16 tiles: These are stored in exactly the same way as 16x16 sprites; that is, the first and second rows have 14 ignored tiles between them. (At least that's how it works when in 16-color mode; I haven't figured out if it's the same for 4 or 256 colors but it's very likely.)

For more information on the tile map format, see BG Basics.

Register $210B/$210C: Character location (2*1B/W)

```
$210B(low) $210C(high)  a: BG1 address  c: BG3 address
bbbbaaaa   ddddcccc     b: BG2 address  d: BG4 address
```

This register selects the location in VRAM where the tile map starts. The byte address is calculated by shifting the four bits left by 13 (multiplying by 8192). Note that, since there is only 64K of VRAM, the highest of the four bits must be set to 0.

For more information on storing characters, see The SNES Graphics Format.

Register $2115: Video Port Control (1b/W)

```
i---ffrr  i: 0=increment when $2118 or $2139 is accessed
             1=increment when $2119 or $213A is accessed
          f: full graphic (?)
          r: increment rate:  00=Increment by 2 bytes   (1x1)
                              01=Increment by 64 bytes  (32x32)
                              10=Increment by 128 bytes (64x64)
                              11=Increment by 256 bytes (128x128)
```

This register controls the way data is uploaded to VRAM. The bits in here are a bit weird, but can be useful. When you want to change only the high byte of a series of VRAM locations (register $2116 * 2 + 1), you should set i to 1. When you want to change just the low byte, set i to 0. When you want to write a whole word, you should set i to 0; otherwise, if i=1, writing a word will cause the high byte of the first location to be changed, followed by the low byte of the next location.

The r bits control the number of bytes by which the VRAM address pointer gets incremented upon a read or write (see table.)

Register $2116/$2117: VRAM Address (2b/W)

```
aaaaaaaa aaaaaaaa  a: word address for accessing VRAM
```

This register is used to set the initial address for a VRAM upload or download. Multiply by two to get the byte address. Note that, since there is only 64K of VRAM, the most significant bit must be set to 0 (If the MSB is set, it must be ignored.)

When reading from VRAM, a "dummy read" must be performed after writing to this register; the first value read is supposed to be meaningless. No "dummy write" is required, however.

Register $2118/$2119: VRAM data write (2b/W)

```
dddddddd dddddddd  d: data to write to VRAM
```

When written, this register writes a byte or word to VRAM. The address is incremented (or not, as the case may be) according to register $2115.

Register $2139/$213A: VRAM data read (2b/R)

```
dddddddd dddddddd  d: data read from VRAM
```

When read from, this register downloads a byte or word from VRAM. The address is incremented according to the settings of register $2115.

Video Control Registers

Register $2100: Screen display register (1b/W)

```
d---bbbb  d: disable screen (if 1, nothing should be displayed?)
          b: brightness. (0=almost black, 15=normal brightness)
```

This register is used for screen fades.

Register $2105: Screen mode register (1b/W)

```
dcbapmmm  d: BG4 tile size  c: BG3 tile size  b: BG2 tile size
          a: BG1 tile size  Sizes are: 0=8x8; 1=16x16. (See reg. $2107)
          p: order of BG priorities  m: General screen mode
```

This register determines the size of tile represented by one entry in the tile map array, the order that BGs are drawn on the screen, and the screen mode. The screen modes are:

```
MODE   # of BGs  Max Colors/Tile   Palettes       Colors Total
0      4         4                 32 (8 per BG)  128 (32 per BG*4 BGs)
1      3         BG1/BG2:16 BG3:4  8              BG1/BG2:128 BG3:32
2      2         16                8              128
3      2         BG1:256 BG2:16    BG1:1 BG2:8    BG1:256 BG2:128
4      2         BG1:256 BG2:4     BG1:1 BG2:8    BG1:256 BG2:32
5      2         BG1:16 BG2:4      8              BG1:128 BG2:32 (Interlaced mode)
6      1         16                8              128 (Interlaced mode)
7      1         256               1              256
```

Notice that Mode 7 has only one BG. All games which appear to have a Mode 7 screen but more than one BG either use sprites to simulate a BG, or switch video modes midframe via HDMA.

The p (priority bit) affects the order in which BGs are drawn on the screen, as follows ("o" refers to the setting of bit 13 of the tile map):

```
p (Priority) 0             1
Drawn first  BG4, o=0      BG4, o=0
   (Behind)  BG3, o=0      BG3, o=0
      .      Sprites with OAM priority 0 (%00)
      .      BG4, o=1      BG4, o=1
      .      BG3, o=1      OAM pri. 1
      .      OAM pri. 1    BG2, o=0
      .      BG2, o=0      BG1, o=0
      .      BG1, o=0      BG2, o=1
      .      Sprites with OAM priority 2 (%10)
      .      BG2, o=1      BG1, o=1
Drawn last   BG1, o=1      OAM pri. 3
  (in front) OAM pri. 3    BG3, o=1
```

The p bit only works in Mode 1. In all other modes, it is ignored (drawing is performed as if this bit were clear.)

Register $210D to $2114: Scroll Registers (Db/W)

```
210D: BG1 Horizontal Scroll   210E: BG1 Vertical Scroll
210F: BG2 Horizontal Scroll   2110: BG2 Vertical Scroll
2111: BG3 Horizontal Scroll   2112: BG3 Vertical Scroll
2113: BG4 Horizontal Scroll   2114: BG4 Vertical Scroll

mmmmmaaa aaaaaaaa  a: Horizontal/Vertical offset m: Mode 7 only
```

This register must be written twice to write the complete value; every time the register is written to, an internal pointer alternates between changing the high and low bytes. The first time you write to the register, the low byte will be changed. There are 11 bits for the offset; when these bits are set to 0, you will see tile map location 0 on the left/top of the screen. The offset is a pixel value, so adding one will scroll the screen right by one pixel. When the screen is scrolled right (or down) from 0, the rightmost (or bottommost) elements of the tile map will come on from the left (top) of the screen.

If a pixel value is placed in this register that is larger than the width of the BG, a modulus can be performed to determine what the actual pixel will be that is displayed. For example, if the BG1 horizontal pixel value is set to 257, but the width of the BG is 256 pixels, the result will be the same as if it was set to 1.

The m bits are used in Mode 7 only; I don't know what they are for. The only thing I've determined is that the scroll register seems to cause pixel-based scrolling: that is, increasing this register by one will make the screen scroll a slight amount right or down. (256/224 would be one screenful.)

Since there is only one BG in Mode 7, the 'm' bits apply only to registers $210D and $210E.

Register $212C & $212D: Main/Sub Screen Designation (2*1b/W)

```
210D: Main Screen Designation
210F: Sub Screen Designation

---sdcba  s: sprites enable  d:  BG4 enable  c: BG3 enable
                            b: BG2 enable  a: BG1 enable
```

The main screen designation is a way to toggle the BGs through software. I think the Sub Screen designation has something to do with color add/subtract.

Windowing Registers

The SNES' Windowing registers are used for clipping; that is, cutting off portions of the screen. If you run Mario World, you'll notice a "circular opening" on the opening screen. This is done using windowing, which clips off the parts of the screen outside of the window. The clipping is only applied to BG1, BG2 and sprites, so that the big letters "Mario World" (BG3) are always visible. Windowing has only a left and right value, so what Mario World and many other games do is constantly vary the left and right values using HDMA.

Register $2123/$2124: Window mask settings (2*1b/W)

```
High Byte Low Byte  4: Settings for BG4  3: Settings for BG3
44443333  22221111  2: Settings for BG2  1: Settings for BG1

Each BG has four bits, which have the following meanings:
dcba    d: Enable Window 2  c: Clip Window 2 in or out  (0=in,
        b: Enable Window 1  a: Clip Window 1 in or out   1=out)
```

These registers determine which Windows to apply to which BGs, and whether clipping should be performed inside or outside the window. To enable windowing, the appropriate bits in registers $212E and $212F must be set in addition to the bits in these registers.

Register $2125: Window mask settings #2 (1b/W)

```
ccccssss  c: Settings for "color windows"  s: Settings for sprite windows
```

Both of these sets of four bits are formatted the same as registers $2123 and $2124.

This is like the last set of registers except it is for sprites and "color windows". I don't know what color windows are; won't someone explain it?

Register $2126 to $2129: Window position designations (4*1b/W)

```
$2126  Window 1 left  position
$2127  Window 1 right position
$2128  Window 2 left  position
$2129  Window 2 right position

For each of these bytes:
xxxxxxxx  x: pixel position of window left or right position.
```

These regs simply tell the range of the window. The window range is inclusive, so if clipping is IN, and the window range is from pixels 3 to 6, all four pixels will be clipped rather than just pixels 4 and 5. If clipping is out, however, clipping will be from pixels 0 to 2 and 7 to 255 inclusive.

Register $212A and $212B: Mask Logic settings (2*1B/W)

```
                 Parameters for:
$212A  44332211  4: BG4  3: BG3  2: BG2  1: BG1
$212B  ????ccss  c: Color windows    s: Sprites

Each of the two bits for each BG and sprites determines the mask logic as follows:

00  OR     10 XOR
01  AND    11 XNOR
```

Apparently, these registers apply only for BGs and sprites that have both windows enabled at the same time. If only one window is enabled, these bits should be ignored. I'm not sure whether the logic is applied to the visible regions or the clipped regions. For example, if pixels 10-20 and 50-60 are clipped, I don't know whether OR or AND would be used to clip both regions and which would clip neither.

Register $212E and $212F: Window designation (2*1b/W)

```
$212E  Main-screen window designation
$212F   Sub-screen window designation
For each of these:
---sdcba  s: sprites enable  d: BG4 enable  c: BG3 enable
                             b: BG2 enable  a: BG1 enable
```

These registers are formatted the same as the visibility designation registers $212C and $212D. A bit must be set in both $212C and $212E or $212D or $212E in order for windows to be enabled for that layer (BG or sprites).

Counter/IRQ/NMI Registers

Register $2137: Software latch for h/v counter (1b/R)

This register does not return a useful value. Apparently, though, you have to read from this register before attempting to read $213C or $213D.

Register $213C & $213D: Horizontal/Vertical scan location (Db/R)

```
For each of the registers:
-------l llllllll  l: scanline numberr ($213C) or horizontal position ($213D)
* vertical range is 0 to 261, 0=top of screen (values over 0-223/0-238 are in the vblank)
* horizontal range is from 0 to 339, 0=far-left of screen.
```

This register tells where exactly on the screen the SNES is sending data to the TV. For the horizontal counter, apparently, zSNES always alternates between sending a strange value (most often 8) and 1, respectively, in register $213C, but only after $2137 has been read once. For the vertical counter, the low byte of the scanline is sent, followed by the high byte, and afterwards nothing but zeros (until $2137 is read again.) One doc suggests register $213F is related to this register, but I don't know how so.

Register $4200: Counter Enable (1b/W)

```
n-vh---j  n: NMI Interrupt enable  v: vertical counter enable
          h: horizontal counter enable  j: joypad enable
```

n and j must be set to enable NMI interrupts and the joypad. Not sure of the exact effects of bits 4 and 5, but they probably must be set to enable IRQ interrupts.

Register $4207/$4208 & $4209/$420A: Horizontal/Vertical IRQ trigger counters (2*2b/W)

```
For each register:
???????l llllllll  l: location to trigger IRQ
```

When IRQs are enabled and the PPU starts drawing at the specified location, an IRQ is generated. Horizontal IRQs are rarely used, because of the high frequency (every scanline) of the interrupts.

Register $4210: NMI Register (1b/R)

```
n---vvvv  n: nmi flag  v: version number
```

The full behavior of these bits is not known (at least, I don't know). Yoshi doc lists "$5A22" for the v bits, whatever that means. Mario World reads this register at the beginning of its NMI procedure and then discards the value read. zSNES apparently returns a 1 in the n bit when NMIs are enabled AND a vblank is in progress.

Register $4211: IRQ Register (1b/RW?)

```
i-------  i: irq flag
```

The full behavior of these bits is not known (at least, I don't know), so take this description with a big grain of salt. If bits 4/5 of register $4200 are set, then as soon as the H/V counter timer becomes the count value set in registers $4207 to $420A, then an IRQ will be "applied" (unless the CPU interrupt flag is enabled. If this is the case, then the interrupt will be called as soon as the flag is cleared.) The IRQ routine must immediately use SEI to prevent recursive calling, then read from this register to "un-apply" the interrupt. The first read will return 1 in the i bit, then subsequent reads will return 0. What happens to this register when bits 4/5 of register $4200 are *not* set is unclear. Some SNES programs use a loop to read this register (in emu's, often getting stuck in this loop) but why a loop is used is unknown.

Register $4212: Status register IRQ Register (1b/RW?)

```
vh-----j  v: whether in vblank state  h: whether in hblank state
          j: whether joypad is ready to be read
```

This register is pretty simple; it returns whether in a vertical/horizontal blank period (1=yes) and whether the joypad is ready to be read from registers $4218 to $421F. The horizontal blank time lasts from "pixels" 256 to 339. (Not really pixels, since nothing is being drawn.) In zSNES, *apparently*, the joypad becomes ready to be read once every frame, and more specifically, scanline 227 (might not be a constant.) How this bit gets cleared to 0, however, is unknown.

A loop in the "leftover" demo suggests that the joypad bit is backwards; that is, a value of 1 indicates the joypad is *not* ready. I think some other demos suggest the opposite is true, though. Interestingly enough, the demos still work regardless of their treatment of the bit. Hmmm...

Joypad Registers

Register $4218 to $421F: Joypad registers(2*1b/R)

```
$4218/$4219  Joypad 1 data
$421A/$421B  Joypad 2 data
$421C/$421D  Joypad 3 data
$421E/$421F  Joypad 4 data

The two registers for each joypad contain the following:

Low Byte     B: B button  Y: Y button  S: Select button T: Start button
BYSTudlr     u: Up  d: Down  l: Left  r: Right
High Byte    A: A button  X: X button  L: Top-left button R: Top-right button
AXLR????
```

This is the easy way to read the joypad. Simply set bit 0 of register $4200, and the SNES will automatically read the state of all the buttons (probably once every frame) into these registers. These registers may be read more than once; that is, reading these registers does not destroy their values. It is best to wait for bit 0 of register $4212 to be set (set or cleared? I'm not sure) before reading from these registers.

Although there are only two joypad ports, two extra joypads can be accessed through $421C-$421F using a multitap device.

Registers $4016/$4217: Old-style Joypad registers (2*1B/R/W)

```
On read:   $4016 is for Player 1; $4017 is for Player 2.
???????b   b: button state
On write:
????????   ?: strobe activation value
```

These are called the "old-style" joypad registers because they are used like the NES used them. First of all, to use these registers, bit 0 of $4200 must be clear (I suppose to prevent conflict between the SNES's automatic read mechanism and the manual method.) If this bit is set, then these registers simply return whether the joypad is connected (0=Not connected.) Based on what commercial games do, it may be neccessary to write a 0 before this function works.

To use the old-style reading method, bit 0 of register $4200 must be clear (probably to prevent conflict between the manual reading and the SNES' automatic reading.) Like on the NES, two values should be written before reading starts: a 1 followed by a 0. This "activates a strobe in the joypad circuitry." Apparently, but not for certain, it is only necessary to do this with $4016; that is, writing to one register resets both joypads. Once the writing is done, each read from each register will return the state of one button, in the following order:

Read #	Button	Read #	Button
1	B	9	A
2	Y	10	X
3	Select	11	L (Top-left)
4	Start	12	R (Top-right)
5	Up	13	-
6	Down	14	-
7	Left	15	-
8	Right	16	-

1 is returned if the button is currently pressed.

If the pattern was similar to that of the NES, reads 17 to 32 would return the state of joypads 3 and 4. However, zsKnight says that read #17 returns whether or not the joypad is connected (0 if not connected.)

DMA Registers See also DMA Transfer Information.

Register $420B: Start DMA transfer (1b/W)

```
76543210  Bits indicate which DMA transfer(s) to initiate
          (1=Transfer, 0=Do not transfer)
```

After setting up a DMA transfer, a 1 should be written to the bit corresponding to the channel you want to use for the transfer. After the 1 is written, the CPU is paused while the DMA transfer takes place; each byte transferred takes one clock cycle. Note that a DMA and HDMA transfer cannot be done on the same channel: if a DMA transfer is initiated while a HDMA channel is enabled, nothing will happen.

Register $420C: Enable H-DMA transfer (1b/W)

```
76543210  Bits indicate which HDMA channel(s) to enable
          (1=Enable transfers, 0=Disable transfers)
```

After setting up the HDMA transfer(s), a 1 should be written to all the bits corresponding to the channels you want enabled. Every scanline, HDMA transfers will occur automatically.

Register $43?0: DMA control register (1b/W)

```
da-fittt  d:(DMA only)  0=read from CPU memory, write to $21?? registers
                        1=read from $21?? registers, write to CPU memory
          a:(HDMA only) 0=Absolute addressing
                        1=Indirect addressing
          i:fixed CPU memory address (1=fixed; 0=inc/dec depending on bit i)
          f:0=increment CPU memory pointer by 1 after every read/write 1=decrement
          t:DMA/HDMA transfer type
```

The 'i' and 'f' bits I had mixed up for quite some time... actually they were backwards in another doc, and so my emu's graphics were screwed for a long time... Bit 3 means a fixed address, so that the same byte is transferred every time.
This register controls the way DMA transfers take place. The t bits decide the 'mode' of the transfer. To describe how this works, suppose the bytes of data to be transferred are $01 $23 $45 $67 $89, and the register to write to is $2118. Assuming f=0 and i=0:

```
Transfer Type           Order of transfer
000: 1 reg              $01->$2118 $23->$2118 $45->$2118 $67->$2118 $89->$2118
001: 2 regs             $01->$2118 $23->$2119 $45->$2118 $67->$2119 $89->$2118
010: 1 reg write twice  $01->$2118 $23->$2118 $45->$2118 $67->$2118 $89->$2118
011: 2 regs write twice $01->$2118 $23->$2118 $45->$2119 $67->$2119 $89->$2118
100: 4 regs             $01->$2118 $23->$2119 $45->$211A $67->$211B $89->$2118
101, 110, 111  unknown behavior
```

Transfer modes 0 and 2 appear to be the same, but are different in HDMA mode. I may have modes 0 and 2 backwards (i.e. 0=1 reg write twice.).

Register $43?1: DMA Destination Address (1b/W)

```
bbbbbbbb  Add this value to $2100 to get the destination address
          (Notice that this restricts what registers you can send to)
```

This is where data will be sent to during a DMA transfer, unless register $43?0 bit 7 is 1, in which case this will be the source register. In an HDMA transfer, this is *always* the source address.

Register $43?2/$43?3/$43?4: DMA Source Address (3b/W)

```
aaaaaaaa aaaaaaaa aaaaaaaa  Full 24-bit address of DMA transfer source address
```

This is where data will be read from during a DMA transfer, unless register $43?0 bit 7 is 1, in which case this will be the destination address. In an HDMA transfer, this is *always* the source address, since HDMA is unidirectional.

Qwertie's horror story: I fixed tons of graphics glitches when I figured this out: you must increment the value in this register after a DMA transfer. That is, transferring $10 bytes forward starting at $00E000 would cause this register to be $00E010 afterward. Also, zsKnight has implied that DMA transfers wrap on a bank boundary; i.e. $43?4 is not changed during a transfer.

Register $43?5/$43?6/$43?7: Bytes to Transfer (2b or 3b/W)

```
???????? aaaaaaaa aaaaaaaa  a: amount of bytes to transfer (DMA only)
                            In HDMA, purpose is unknown.
```

When using DMA, set this to the number of bytes you want transferred. Note: if set to 0, 65536 bytes of data will be transferred. I am not sure whether this register should be set to 0 after the transfer.

In HDMA, it is not neccessary for a SNES programmer to write to this register. This register probably contains a pointer the next or previous value written. Based on circumstantial evidence from one game, this register may contain the value pointer minus one in regular HDMA (e.g. if this register contained $123456, it would mean that the next value to be written was contained in $123457.) In indirect HDMA, this register probably contains the pointer to the value after indirection.

When using regular HDMA, either the bank is copied to this register or the bank stored here is ignored. In Indirect HDMA, however, the bank address has to be set manually, and the SNES will automatically determine the offset.

Register $43?8/$43?9: HDMA count pointer (2b/RW?)

This register does not have to be directly written by the programmer; the SNES updates it automatically.

I'm calling this register a count pointer because it probably points to the byte in the HDMA table that contains the count value until the next segment of the HDMA table is executed. This applies to both HDMA and Indirect HDMA. (In Indirect HDMA, this register could also be called the "pointer before indirection".)

Since this is a two and not three-byte register, the bank value should be taken from $43?4.

Register $43?A: Scanlines left (1b/RW?)

This register does not have to be directly written by the programmer.

This register is a countdown that contains the number of scanlines remaining until the next segment of a HDMA table is executed. When this counter reaches 0, the pointer contained in register $43?8 is advanced to the next count value in the HDMA table, and this register is loaded with the count value for the next segment of the table.

Register $2180/$2181/$2182/$2183: WRAM access (4b/RW)

```
$2180  dddddddd  d: Data byte
$2181  ???????x xxxxxxxx xxxxxxxx  x: address
```

This isn't a DMA register at all, but is often used in conjunction with DMA to do memory-to-memory or memory-fill operations. This is a fairly simple register, but I'm documenting it here because it's not fully

covered anywhere else. A write or a read to this register causes a read/write to the specified address, and every time a read/write occurs, the address part of the register is incremented by one, including the MSB.

Important: The x bits (address) seem to actually form a RAM address, NOT a CPU address. Therefore, the highest 7 bits of the register are ignored, since the SNES has only 128 KB of RAM.

DMA Transfer Basics

First of all, here's a list of DMA (Direct Memory Access) registers:

$43?0: DMA control register
$43?1: DMA destination register ($21xx)
$43?2/$43?3/$43?4: DMA source address
$43?5/$43?6: Number of bytes to transfer
$420B: Start DMA transfer
$420C: Enable HDMA transfers

There are 8 DMA channels, numbered 0 to 7. Simply replace the question mark with the number of the channel you want to use. Each channel can be used independently of the others.

There are two basic types of DMA: DMA and HDMA. Normal DMA is quite straightforward. The SNES program simply puts a source address into $43?2, a destination address into $43?1 and the number of bytes to copy into $43?5. The reason $43?1 is only a one-byte register is because the high byte is always $21; e.g. if you set it to $18, the transfer destination will be $2118. The DMA control register controls how the bytes will be transferred. Using the previous example, setting it to one(1) will cause bytes written to alternate between $2118 and $2119. Use $43?0 to start a transfer. For instance, if you were using DMA channel 1 ($4310, etc.), you would set bit one in that register. (e.g. LDA #$2 \ STA $420B.)

HDMA is a powerful, and largely undocumented, tool for generating special effects. It is used often by commercial games and demos alike. Its most common use is to change one or more of the scroll registers mid-frame; some games also change the video mode or palette mid-frame with it. It is set up similar to DMA, except that $43?5 is ignored and probably, bits 3, 4, 5 and 7 of $43?0 are ignored.

Normal HDMA

There are two types of HDMA, controlled by bit 6 of register $43?0. "Normal" HDMA is the simplest.

The source address register points to an "HDMA table". Such a table consists of a list of "delay counts" and "write values". For example, consider the following HDMA table, which is an abridgement of what I use in my demo QwertEmu.smc. It the destination is the BG1 scroll register.

```
hdma_table:
        .dcb 3 : .dcw 1    ; This makes the cool wavey BG effect.
        .dcb 4 : .dcw 2
        .dcb 6 : .dcw 3
        .dcb 7 : .dcw 4
        .dcb 6 : .dcw 3
        .dcb 4 : .dcw 2
        .dcb 3 : .dcw 1
        .dcb 2 : .dcw 0
        .dcb 3 : .dcw 511
        .dcb 4 : .dcw 510
        .dcb 6 : .dcw 509
        .dcb 7 : .dcw 508
        .dcb 6 : .dcw 509
        .dcb 4 : .dcw 510
        .dcb 3 : .dcw 511
```

```
.dcb 2 : .dcw 0
.dcb 0
```

The first value is the scanline count, and the second is the value to write. So, after I set up the addresses and set $420C to 1 to enable the HDMA transfer, the SNES (I think) will wait until the beginning of the next frame before executing this transfer. In the example above, two bytes are written, because the transfer is using mode 2 (i.e. 1 reg write twice, which is the proper way to set a scroll register.) The first byte written is the LSByte (1) followed by the MSByte (0). The first value is written in scanline 0; the first value is written at scanline 0 regardless of the count value. After this, the SNES waits three more scanlines (according to the count value of 3) before writing a $0002 to the scroll register. Eventually, the SNES will come across the wait value of $0, which indicates to the SNES that the table is over and the transfering should stop. The SNES will automatically restart the HDMA transfer at the beginning of the table next frame.

Note: Transfers probably do not stop at the VBlank period. That is, if the table is so long that it lasts into the VBlank period, transferring will still continue (this is unconfirmed though). If the table is so long that the next frame starts before a $0 has been encountered, the SNES will reset the transfer from the beginning anyway.

Indirect HDMA

Most of the properties of regular HDMA also apply to Indirect HDMA: Indirect HDMA has a count value, the transfer modes are the same, and Indirect HDMA tables are also reset to the beginning at the beginning of each frame.

An Indirect HDMA table is formatted like this:

Count Byte/Address Offset/Count Byte/Address Offset/Count Byte/Address Offset...

So, instead of having the values to write directly located in the HDMA table, they are accessed via the Address Offset pointer. Let's go through an example. If register $43?2 is set to $1289AB, and register $43?1 is set to $05, here is the table (this is similar to a table used by Metroid 3 in the opening level):

```
Memory at $1289AB (Hex)
1F 01 03 57 02 03 00
```

The second and third bytes are used as pointers to values to write. The bank value after indirection must be set manually by the SNES program via register $4307. Metroid 3 uses a bank value of $12 so the table points to $120301 and $120302, respectively. If the memory contained at $120301 is (Hex):

```
09 07
```

The $9 will be written at the beginning of scanline 0, and after $1F (31) scanlines, $7 will be written. This makes sense when you run the game; the status bar at the top is shown in video mode 1, while below it is a mode 7 screen.

If you think using HDMA in this way is rather useless, you are correct. There's no reason this couldn't be done using normal HDMA, but many games use indirect HDMA anyway. The nice thing about it is that you can store the count values and pointers in ROM, and the actual values to write (which may be variable) in RAM. Thus the same basic ROM table can be re-used with different data sets.

Continuous Mode

This mode adds an extremely annoying twist to the job of writing an emulator. It can be used in both normal and indirect HDMA. To use this mode, set bit 7 of the count value to 1.*

Continuous mode means that a value is written every single scanline rather than waiting for the count value to expire. I'll provide two examples, one for normal and one for indirect HDMA. Assume the transfer mode is 1

(One byte write once.) If your count value is 4, then in normal HDMA, a segment of your table would look something like this:

```
84 01 02 03 04
^  ^^-^^-^^-^^--These are the values to write
 --This is the count value with a continuous mode flag
```

But in an Indirect HDMA version of this, the table segment would look something like:

```
84 34 12
```

And the memory at $1234 would contain:

```
01 02 03 04
```

* There may be an exception. If the count value is exactly $80, it may not mean to use continuous mode, but rather have a count value of 128.

HDMA Priorities

Lower HDMA channel numbers have a high priority. That is, if you have two tables running at the same time:

```
hdma_table_channel_0:
        .dcb 10 : .dcb 1    ; Suppose that this table is used to set the
        .dcb 0              ; palette index ($2121) to 1 using mode 0
                            ; N.B.: depending on the mode used, the value
                            ; to transfer may be a byte, word or dword.  In
                            ; mode 0, only a byte is transferred at once.
hdma_table_channel_1:
        .dcb 10 :.dcw $8FFF ; Suppose that this table is used to set the
        .dcb 0              ; palette value ($2122) using mode 2
```

In this example, at scanline #9, the palette index register will be set *before* the palette value is written. If you reversed the channel numbers, however, the transfer might not occur correctly.

Please note that some of the HDMA information is inaccurate, as evidenced by the fact that my emu doesn't emulate it properly!

Gad! That took long enough to write.

Check out my page! http://www.geocities.com/SiliconValley/Bay/6633 or maybe http://members.xoom.com/Qwertie.